

5.2 Internal Working of Spring Security, Part 1

• Adding Spring Security :

- In a Spring Boot application, simply adding the **spring-boot-starter-security** dependency enables Spring Security.

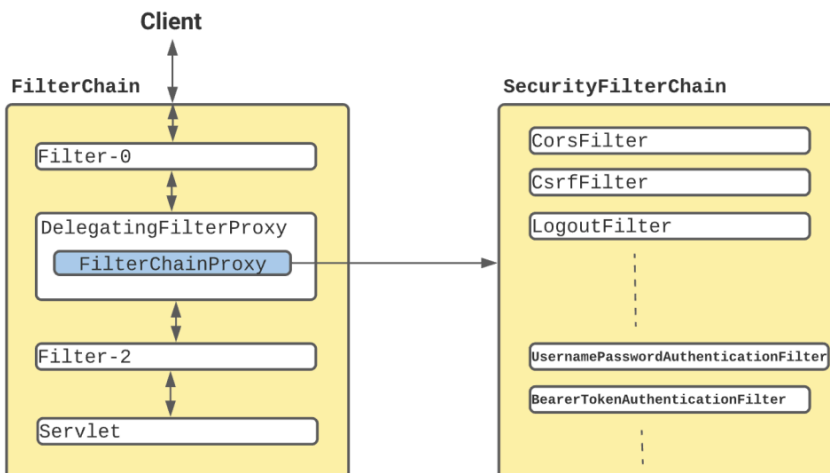
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

- Spring Boot automatically configures a default security setup with sensible defaults, including:
 - authentication for all endpoints
 - a default login page
 - CSRF protection
 - common security headers.

• Authentication and Authorization:

- **Authentication** is the process of verifying the identity of a user. It ensures that the user is who they claim to be. Authentication typically involves validating credentials (such as a username and password, OTP, or OAuth login) and establishing the user's authenticated security context.
- **Authorization** is the process of determining whether an authenticated user has the necessary permissions to access a particular resource or perform an action. It controls what an authenticated user can or cannot do.
- In a single line can say, **Authentication** answers "Who are you?" while **Authorization** answers "What are you allowed to do?".

• Internal working of Spring-Security:



- In a Spring Boot application, **SecurityFilterAutoConfiguration** automatically registers the **DelegatingFilterProxy** filter with the name **springSecurityFilterChain**.
- Once the request reaches the **DelegatingFilterProxy**, it delegates processing to the **FilterChainProxy**. The **FilterChainProxy** selects the appropriate **SecurityFilterChain** for the incoming request and executes its configured security filters before the request reaches the **DispatcherServlet**.

5.2 Internal Working of Spring Security, Part 1

• Default behaviour of Spring-Security :

- Creates a bean named **springSecurityFilterChain** (a **FilterChainProxy**) and registers a **DelegatingFilterProxy** with the **servlet container**, which delegates every request to the **springSecurityFilterChain** bean.
- Enables **HTTP** Basic authentication and a default form-based login for browser applications.
- Generates a default **login page**.
- Creates a **default user** (user) with a randomly generated **password**, which is printed to the application logs (**console**).
- Supports secure password hashing using **BCryptPasswordEncoder** (recommended for application users).
- Spring enables **/logout** by default.
- Enables additional security features such as:
 - **CSRF** protection
 - **Session fixation** protection
 - Common **security HTTP** headers

